

Integrated Machine Learning Project Using School Dataset

Project Summary

This project uses a School Dataset to perform prediction, classification, and pattern analysis using different Machine Learning algorithms. The project helps in analyzing student performance, school infrastructure, and educational facilities.

The algorithms used are:

1. Decision Tree
 2. Simple Linear Regression
 3. Multiple Regression
 4. Logistic Regression
 5. Support Vector Machine (SVM)
 6. K-Nearest Neighbor (KNN)
 7. Naïve Bayes
 8. Association Rule Algorithm
-

Overall Objectives

- To analyze educational data using Machine Learning.
- To predict student performance.
- To classify schools based on different factors.
- To identify hidden educational patterns.
- To compare different Machine Learning models.

Dataset Features

Example columns used from school dataset:

- Teacher_Student_Ratio
- Avg_Teacher_Experience_Years
- Parent_Literacy_Rate
- Internet_Available
- Electricity_Available
- %_Math_Score
- %_Science_Score
- Urban_Rural

1. Decision Tree Algorithm

Project Title

Prediction of School Performance Using Decision Tree

Objective

To classify schools as High Performance or Low Performance.

Input Variables

- Teacher_Student_Ratio
- Parent_Literacy_Rate
- Internet_Available

Target Variable

- Class_Performance

Python Code

```
import pandas as pd
# Load Dataset
df = pd.read_csv("School_data.csv")
print(df.head())
# Data Preprocessing
df = df.dropna()
import numpy as np
df = pd.read_csv('School_data.csv')
df['Performance_Class'] = np.where(df['%_Math_Score'] >= 60, 1, 0)
```

```
# ===== Decision Tree Algorithm =====

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = df[['Teacher_Student_Ratio','Parent_Literacy_Rate',
        'Internet_Available','Avg_Teacher_Experience_Years']]
y = df['Performance_Class']

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train,y_train)

pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test,pred))
```

Accuracy : 0.5057692307692307

Expected Output

Predicts whether school performance is high or low.

Conclusion

Decision Tree identifies important factors affecting school performance.

2. Simple Linear Regression

Project Title

Math Score Prediction Using Parent Literacy Rate

Objective

To predict Math Score based on Parent Literacy Rate.

Independent Variable

- Parent_Literacy_Rate

Dependent Variable

- %_Math_Score

Python Code

```
# ===== Simple Linear Regression =====

from sklearn.linear_model import LinearRegression
# Load dataset
df = pd.read_csv('School_data.csv')
# Fill missing values
df['Parent_Literacy_Rate'] = df['Parent_Literacy_Rate'].fillna(
    df['Parent_Literacy_Rate'].mean()
)
df['%_Math_Score'] = df['%_Math_Score'].fillna(
    df['%_Math_Score'].mean()
)
# Features and target
X = df[['Parent_Literacy_Rate']]
y = df['%_Math_Score']
# Train model
```

```
model = LinearRegression()
model.fit(X, y)
# Correct prediction input
sample_data = pd.DataFrame({
    'Parent_Literacy_Rate': [70]
})
# Prediction
prediction = model.predict(sample_data)

print("Prediction:", prediction)
```

Prediction: [60.2705231]

Expected Output

Shows relationship between literacy and math score.

Conclusion

Higher parent literacy improves student performance.

3. Multiple Regression

Project Title

Science Score Prediction Using Multiple Factors

Objective

To predict Science Score using multiple variables.

Input Variables

- Teacher_Student_Ratio
- Parent_Literacy_Rate
- Teacher_Experience

Target Variable

- %_Science_Score

Python Code

```
# ===== Multiple Linear Regression =====

from sklearn.linear_model import LinearRegression
# Load dataset
df = pd.read_csv("School_data.csv")
# Fill missing values
df = df.fillna(df.mean(numeric_only=True))
# Features
X = df[['Teacher_Student_Ratio',
        'Parent_Literacy_Rate',
        'Avg_Teacher_Experience_Years']]
# Target
y = df['%_Science_Score']
# Train model
model = LinearRegression()
model.fit(X, y)
```

```
# Proper prediction input
sample_data = pd.DataFrame({
    'Teacher_Student_Ratio': [30],
    'Parent_Literacy_Rate': [70],
    'Avg_Teacher_Experience_Years': [10]
})
# Predict
prediction = model.predict(sample_data)

print("Prediction:", prediction)
```

Prediction: [61.22598607]

Expected Output

Predicts science score using multiple educational factors.

Conclusion

Multiple Regression gives more accurate predictions.

4. Logistic Regression

Project Title

Internet Availability Prediction

Objective

To predict whether schools have internet facilities.

Input Variables

- Electricity_Available
- Parent_Literacy_Rate

Target Variable

- Internet_Available

Python Code

```
# ===== Logistic Regression =====

from sklearn.linear_model import LogisticRegression
# Load dataset
df = pd.read_csv('School_data.csv')
# Fill missing values
df = df.fillna(df.mean(numeric_only=True))
# Create classification target
df['Performance_Class'] = (df['%_Math_Score'] >= 60).astype(int)
# Features
X = df[['Internet_Available',
        'Parent_Literacy_Rate'
        ]]
# Target
y = df['Performance_Class']
# Create model
```

```
model = LogisticRegression()  
# Train model  
model.fit(X, y)  
# Predict  
print(model.predict(X.head()))
```

Output : [1 1 1 1 1]

Expected Output

Predicts internet availability.

Conclusion

Logistic Regression works well for binary classification.

5. Support Vector Machine (SVM)

Project Title

Urban and Rural School Classification

Objective

To classify schools into Urban or Rural categories.

Python Code

```
# ===== Support Vector Machine =====

import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('School_data.csv')

# Fill missing values
df = df.fillna(df.mean(numeric_only=True))

# Create target column for Urban/Rural Classification
# Assuming:
# 1 = Urban
# 0 = Rural
# Example using Internet availability
df['Area_Class'] = (df['Internet_Available'] == 1).astype(int)

# Features
X = df[['Teacher_Student_Ratio',
        'Parent_Literacy_Rate',
        'Avg_Teacher_Experience_Years']]

# Target
y = df['Area_Class']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```

```

    test_size=0.2,
    random_state=42
)
# Create SVM model
model = SVC(kernel='linear')
# Train model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Predict sample values
sample_data = pd.DataFrame({
    'Teacher_Student_Ratio': [25],
    'Parent_Literacy_Rate': [80],
    'Avg_Teacher_Experience_Years': [12]
})
prediction = model.predict(sample_data)
# Display result
if prediction[0] == 1:
    print("Predicted Area: Urban")
else:
    print("Predicted Area: Rural")

```

Accuracy: 0.7278846153846154

Predicted Area: Rural

Expected Output

Classifies schools as Urban or Rural.

Conclusion

SVM provides high classification accuracy.

6. K-Nearest Neighbor (KNN)

Project Title

School Performance Prediction Using KNN

Objective

To classify schools using nearest neighbors.

Python Code

```
# ===== KNN Algorithm =====

import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_csv('School_data.csv')

# Fill missing values
df = df.fillna(df.mean(numeric_only=True))

# Create target
df['Performance_Class'] = (df['_Math_Score'] >= 60).astype(int)

# Features and target
X = df[['Teacher_Student_Ratio',
        'Parent_Literacy_Rate']]
y = df['Performance_Class']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = KNeighborsClassifier(n_neighbors=5)

# Train
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy: 0.5134615384615384

Expected Output

Predicts school performance.

Conclusion

KNN compares similar schools for prediction.

7. Naïve Bayes Algorithm

Project Title

Library Availability Prediction

Objective

To predict whether a school has a library.

Python Code

```
# ===== Naive Bayes Algorithm =====  
  
from sklearn.naive_bayes import GaussianNB  
X = df[['Internet_Available','Functional_Toilets',  
        'Library_Available']]  
y = df['Performance_Class']  
model = GaussianNB()  
model.fit(X,y)  
print(model.predict(X.head()))
```

Output : [1 1 1 1 1]

Expected Output

Predicts library availability.

Conclusion

Naïve Bayes is simple and efficient for predictions.

8. Association Rule Algorithm

Project Title

Educational Pattern Analysis

Objective

To identify relationships between school facilities and student performance.

Python Code

```
# ===== Association Rule Algorithm =====  
import pandas as pd  
from mlxtend.frequent_patterns import apriori, association_rules  
# Load dataset  
df = pd.read_csv('School_data.csv')  
# Fill missing values  
df = df.fillna(df.mean(numeric_only=True))  
# Select columns  
basket = df[['Electricity_Available',  
             'Internet_Available',  
             'Library_Available']]  
# Convert to Boolean  
basket = basket.astype(bool)  
# Apply Apriori  
freq = apriori(  
    basket,  
    min_support=0.2,  
    use_colnames=True  
)  
# Generate rules  
rules = association_rules(  
    freq,  
    metric='confidence',
```

```

    min_threshold=0.5
)
# Display output
print(rules.head())

```

Output:

Antecedents	Consequents	Support	Confidence	Lift	Jaccard
Internet_Available	Electricity_Available	0.292500	0.884817	0.987985	0.313285
Library_Available	Electricity_Available	0.432692	0.894988	0.999342	0.457224

Antecedents	Consequent Support	Leverage	Conviction	Zhang's Metric	Kulczynski
Internet_Available	0.895577	-0.003557	0.906582	-0.017842	0.605711
Library_Available	0.895577	-0.000285	0.994392	-0.001272	0.689066

Expected Output

Finds hidden educational patterns.

Conclusion

Association Rules help in policy-making and planning.

Tools and Technologies Used

- Python
- Pandas
- NumPy
- Scikit-learn
- Matplotlib
- Jupyter Notebook

Final Conclusion

This project successfully demonstrates how Machine Learning algorithms can be used in educational data analysis. Different algorithms help in prediction, classification, and discovering useful educational patterns. These models can support schools and government organizations in improving educational quality and decision-making.